

In this assignment, you will perform bioinformatic analyses of gene expression data using several powerful tools in the field of transcriptomics. The goal of this exercise is to provide hands-on experience with the processing, quantification, and analysis of RNA-Seq data, which is essential for understanding gene expression levels across different conditions.

You will be working with tools such as Trimmomatic, gffread, Kallisto, and Bowtie2, which are commonly used in modern bioinformatics pipelines. By the end of this task, you will gain practical skills in filtering raw sequencing data, creating reference transcriptomes, quantifying gene expression, and performing downstream statistical analysis.

CONNECTION TO THE SERVER

- Please contact our administrators for the VPN access.
- Once in the VPN, you can connect to the server using:

```
$ ssh studdatr@143.93.91.124
password: studdatr_2024
```

WORKING ENVIRONMENT

You can work on this task either locally on your own machine or on the bioserver. However, the bioserver already has all the required software installed.

Both options are valid, so you can choose the one that best suits your needs. Ensure that you organize and save your files according to the structure mentioned below, regardless of where you run your analysis.

If you prefer to work on your local machine, you will need to install the required tools:

- Trimmomatic: Download [Trimmomatic](#) and follow the installation instructions.
- gffread: You can install gffread from the [gffread GitHub repository](#).
- Kallisto: Install Kallisto following the instructions on the [Kallisto GitHub page](#).
- Bowtie2: Install Bowtie2 using [this instructions](#) or download it from the [Bowtie2 website](#).

If you work in the bioserver, please create a directory inside `/media/BioNAS/ag_hallab/DATR/` using your last name. Inside your directory, create the standard scientific structure:

```
LastName/
|-- material/
|-- methods/
|-- results/
|-- README.md
```

- **material:** This folder should contain any data files or resources you will be using for your analysis (e.g., raw data, external datasets, input files).
- **methods:** This folder should contain scripts, examples, instructions.
- **results:** This folder should contain the results of your analysis, including any output files like processed data, plots, and figures.
- Use the **README** file as your Labbook: A student should be able to reproduce your results reading this document. Note that it should be only as long as required, i.e. the content can be short; reproducibility is the key topic here.

Your analysis must be carried out using our job-queue (see below) and all required software should be preinstalled. If you miss some software, please contact us immediately.

Run a job using the job-queue system SLURM

[Slurm](#) is a very efficient system to manage resources and jobs in a high compute cluster environment. It is one of the most widespread job-queue-systems used. Basically, you can execute any job that you can run from your terminal. To understand how it works and how it is used, please do the following:

Read - especially the comments in - the example script:

```
cd /media/BioNAS/ag_hallab/DATR/methods
less -S example_slurm_script.sh
```

Copy the example script to your LastName folder:

```
cd <LastName>
mkdir methods
cd methods
cp /media/BioNAS/ag_hallab/DATR/methods example_slurm_script.sh .
```

Now run the example script with:

```
sbatch example_slurm_script.sh
```

Monitor the job status with:

```
squeue
```

Wait until the job has finished and now have a look at the files generated by the job you just ran.

Implement your project analysis and use a SLURM script to run it.

DATA

In this assignment, you will work with RNA-seq data to explore the cellular stress response in *Saccharomyces cerevisiae* (yeast). The data is sourced from the study:

Nuño-Cabanes, C., Ugidos, M., Tarazona, S. et al. *A multi-omics dataset of heat-shock response in the yeast RNA binding protein Mip6*. *Sci Data* 7, 69 (2020). <https://doi.org/10.1038/s41597-020-0412-z>

In this study, researchers investigated the heat-shock response of yeast, focusing on the RNA binding protein Mip6. They employed a multi-omics approach to examine both transcriptomic and proteomic changes across different stress conditions, with multiple biological replicates to ensure data reliability.

For this task, we will use three biological replicates of RNA-seq for each experimental condition, which is essential for evaluating both inter- and intra-group variability.

Experimental Conditions:

- Control: SRA Accessions SRR9929273,SRR9929263,SRR9929264 — Three biological replicates measured at 30°C.
- Condition 1: SRA Accessions SRR9929265,SRR9929271,SRR9929274 — Three biological replicates measured at 39°C, 20 minutes post-heat shock.
- Condition 2: SRA Accessions SRR9929281,SRR9929279,SRR9929272 — Three biological replicates measured at 30°C, 120 minutes post-heat shock.

DATA LOCATION

The RNA-seq data files for this assignment are stored in the following directory:

```
/media/BioNAS/ag_hallab/DATR/material
```

The data for each biological replicate is stored in two files, given that it is paired-end sequencing. Each replicate includes:

- _1: the first read (or forward end)
- _2: the second read (or reverse end)

REFERENCE DATA

For this analysis, we are using the S288C reference genome of *Saccharomyces cerevisiae*. This genome was chosen because it is a widely used and well-annotated strain of yeast, specifically the strain S288C.

The reference genome is obtained from the [Saccharomyces Genome Database \(SGD\)](#), which provides the complete genomic sequence and functional annotations. The data files used for

creating the reference transcriptome include the genomic sequence (in FASTA format) and the genomic annotations (in GFF format). The annotations detail the locations of genes, exons, and other features of the genome. However, the files need slight modifications to be compatible with gffread for creating a proper reference transcriptome.

- The chromosome names in the FASTA file (for example, "ref|NC_001138|") were modified to match the standard chrI, chrII, ..., chrXVI naming convention.
- Some chromosome entries like chrmt were not present in the original FASTA, but were added based on the GFF annotations as an empty sequence.

These modifications were done to ensure gffread could process the data without errors.

The reference genome data can be found on the server in the following directory:

- GFF: /media/BioNAS/ag_hallab/DATR/material/reference.gff
- Original Fasta file: /media/BioNAS/ag_hallab/DATR/material/original_reference.fsa
- Modified fasta file: /media/BioNAS/ag_hallab/DATR/material/reference.fsa

EXERCISES

1. Create the Reference Transcriptome with GFFread

Use GFFread to generate the reference transcriptome. This process uses the GFF file (genome annotation) and the FASTA file (genome sequence) to build a transcriptome file. This transcriptome is used for gene expression quantification.

Files Needed:

- GFF file: /media/BioNAS/ag_hallab/DATR/material/reference.gff
- FASTA file: /media/BioNAS/ag_hallab/DATR/material/reference.fsa

Command to create the reference transcriptome. Remember to use SLURM:

```
gffread /media/BioNAS/ag_hallab/DATR/material/reference.gff \  
-g /media/BioNAS/ag_hallab/DATR/material/reference.fsa \  
-w /your_dir/results/output_transcriptome.fa
```

- The -g option specifies the FASTA reference genome.
- The -w option defines the output FASTA file (reference transcriptome).

This will generate a reference transcriptome in FASTA format (output_transcriptome.fa).

2. Quality Filtering with Trimmomatic

Use Trimmomatic to filter the quality of the raw reads (FASTQ files). Trimmomatic will remove adapter sequences, low-quality bases, and other contaminants.

Files Needed:

- Control replicates
- Condition1 replicates
- Condition2 replicates

Command for Trimmomatic. Remember to use SLURM:

```
java -jar /usr/local/bin/trimmomatic/trimmomatic-0.39.jar PE -threads 4 \  
/media/BioNAS/ag_hallab/DATR/material/control/SRR9929273_1.fastq \  
/media/BioNAS/ag_hallab/DATR/material/control/SRR9929273_2.fastq \  
<your_dir>/material/control/SRR9929273_1T.fq.gz /dev/null \  
<your_dir>/material/control/SRR9929273_2T.fq.gz /dev/null \  
ILLUMINACLIP:/usr/local/bin/trimmomatic/adapters/TruSeq3-PE.fa:2:30:10:2:  
keepBothReads MINLEN:70
```

- `java -jar /usr/local/bin/trimmomatic/trimmomatic-0.39.jar PE -threads 4`: Executes the Trimmomatic program with 4 threads for parallel processing on paired-end data.
- `/media/BioNAS/ag_hallab/DATR/material/control/SRR9929273_1.fastq`: Input file for the first read of paired-end data.
- `/media/BioNAS/ag_hallab/DATR/material/control/SRR9929273_2.fastq`: Input file for the second read of paired-end data.
- `<your_dir>/material/control/SRR9929273_1T.fq.gz`: Output file for the first read after trimming (compressed).
- `<your_dir>/material/control/SRR9929273_2T.fq.gz`: Output file for the second read after trimming (compressed).
- `ILLUMINACLIP:/usr/local/bin/trimmomatic/adapters/TruSeq3-PE.fa:2:30:10:2`: Adapter trimming using the specified adapter file `TruSeq3-PE.fa`, allowing a minimum of 2 matching bases, a quality threshold of 30, and a maximum of 10 mismatches.
- `keepBothReads`: Both reads from each pair are kept, even if one is discarded due to low quality.
- `MINLEN:70`: Discards any reads shorter than 70 bases after trimming.

This command ensures that both paired reads are trimmed for adapters and low-quality bases, while maintaining high-quality reads with a minimum length of 70 bases. The resulting files are stored in compressed FASTQ format.

Repeat this command for all conditions and replicates.

3. Quantify Gene Expression with Kallisto and Bowtie2

Use Kallisto (alignment-free) and Bowtie2 (alignment-based) to quantify gene expression from the FASTQ reads.

Files Needed:

- Kallisto:
 - The transcriptome of reference (generated with GFFread).
 - The trimmed FASTQ files.
- Bowtie2:
 - The FASTA reference genome.
 - The trimmed FASTQ files.

Create an index of the reference transcriptome for Kallisto. Remember to use SLURM:

```
kallisto index -i <your_dir>/results/transcriptome.idx \  
<your_dir>/results/<gffread_output.fa>
```

- `kallisto index -i <your_dir>/results/transcriptome.idx`: Creates an index of the transcriptome using the Kallisto tool. The `-i` option specifies the path where the index file should be stored.
- `<your_dir>/results/<gffread_output.fa>`: This is the input file containing the transcript sequences in FASTA format, which was generated using the `gffread` tool. It contains the reference transcriptome from which the index will be built.

Quantify gene expression with Kallisto. Remember to use SLURM:

```
kallisto quant -i <your_dir>/results/transcriptome.idx \  
-o results/<condition> -b 100 <your_dir>/material/control/SRR9929273_1T.fq.gz \  
<your_dir>/material/control/SRR9929273_2T.fq.gz
```

- `kallisto quant -i <your_dir>/results/transcriptome.idx`: This part specifies the input index file that was created earlier with the `kallisto index` command. The `-i` option points to the location of the pre-built transcriptome index.
- `-o results/<condition>`: This option specifies the directory where the results will be saved. The folder name should correspond to the experimental condition, which helps in organizing results from different conditions.
- `-b 100`: This option specifies that 100 bootstrap samples should be generated for estimating the uncertainty in the abundance estimates.

- `<your_dir>/material/control/SRR9929273_1T.fq.gz`: This is the input file for the first paired-end read (forward read) in compressed FASTQ format.
- `<your_dir>/material/control/SRR9929273_2T.fq.gz`: This is the input file for the second paired-end read (reverse read) in compressed FASTQ format.

Repeat the Kallisto command for each condition and replicate.

Create an index for Bowtie2. Remember to use SLURM:

```
bowtie2-build /media/BioNAS/ag_hallab/DATR/material/reference.fsa \
<your_dir>/results/bowtie2_index
```

- `bowtie2-build`: This is the command to build an index for Bowtie2 from a reference genome file. It creates a set of index files that Bowtie2 will use to align RNA-seq reads.
- `/media/BioNAS/ag_hallab/DATR/material/reference.fsa`: This is the input reference genome file in FASTA format. The file contains the reference sequences (e.g., chromosome sequences) that will be used for aligning reads.
- `<your_dir>/results/bowtie2_index`: This specifies the output prefix for the generated index files. Bowtie2 will create multiple index files, all prefixed with the given name (e.g., `<your_dir>/results/bowtie2_index.*`).

Run alignment. Remember to use SLURM:

```
bowtie2 -x <your_dir>/results/bowtie2_index -1 \
<your_dir>/material/control/SRR9929273_1T.fq.gz -2 \
<your_dir>/material/control/SRR9929273_2T.fq.gz -S \
<your_dir>/results/<condition>/output_bowtie2.sam
```

- `bowtie2`: The main command to run the Bowtie2 aligner, which is used for aligning sequencing reads (either single-end or paired-end) to a reference genome or transcriptome.
- `-x <your_dir>/results/bowtie2_index`: This specifies the Bowtie2 index prefix that was previously created using the `bowtie2-build` command. This index is used for aligning the input reads to the reference genome.
- `-1 <your_dir>/material/control/SRR9929273_1T.fq.gz`: This is the file containing the first set of paired-end reads. The `-1` flag indicates that these are the first reads in a paired-end alignment.
- `-2 <your_dir>/material/control/SRR9929273_2T.fq.gz`: This is the file containing the second set of paired-end reads. The `-2` flag indicates that these are the second reads in a paired-end alignment.

- `-S <your_dir>/results/<condition>/output_bowtie2.sam`: This specifies the output file in SAM format, where the results of the read alignment will be saved. The `-S` flag is used to specify the output in SAM format, which is a common format for storing alignment results.

Repeat the Bowtie2 command for each condition and replicate.

Convert .sam to .bam. Remember to use SLURM:

```
samtools view -bS <your_dir>/results/<condition>/output_bowtie2.sam \
| samtools sort -o <your_dir>/results/<condition>/alignment_sorted.bam
```

- `samtools view -bS <your_dir>/results/<condition>/output_bowtie2.sam`: This command converts the SAM file into a BAM file (binary format). The `-b` flag specifies that the output should be in BAM format, while the `-S` flag indicates that the input is in SAM format.
- `|`: This is a pipe operator, which passes the output from one command as input to the next. In this case, the output of `samtools view` is passed as input to `samtools sort`.
- `samtools sort -o <your_dir>/results/<condition>/alignment_sorted.bam`: This command sorts the BAM file generated by the previous step. The `samtools sort` command sorts the reads based on their genomic position and writes the sorted result to a new BAM file. The `-o` flag specifies the output file name.

Repeat command for each condition and replicate.

Generate gene counts. Remember to use SLURM:

```
featureCounts -F GFF -p -t gene -g ID -a \
/media/BioNAS/ag_hallab/DATR/material/reference.gff \
-o <your_dir>/results/<condition>/counts.txt \
<your_dir>/results/<condition>/alignment_sorted.bam
```

- `-F GFF`: Specifies that the annotation file is in GFF (General Feature Format) format.
- `-p`: Indicates that the data is paired-end, so the reads come in pairs.
- `-t gene`: Defines the feature type to be counted as `gene`, meaning only reads aligned to genes are counted.
- `-g ID`: Specifies that the attribute `ID` should be used as the gene identifier.
- `-a /media/BioNAS/ag_hallab/DATR/material/reference.gff`: Provides the GFF annotation file for `featureCounts` to reference, which contains the gene locations and IDs.

- `-o <your_dir>/results/<condition>/counts.txt`: This flag specifies the output file path where the read counts will be saved. The result will be a tab-separated file, where each row corresponds to a gene, with the corresponding read counts.
- `<your_dir>/results/<condition>/alignment_sorted.bam`: This is the input file, which is the BAM file containing the aligned reads that will be counted.

Repeat command for each condition and replicate.

4. Extract TPM values

- For Kallisto, TPM values are automatically included in the output files (`abundance.tsv`) within the output folder. If you want to perform the analysis at the gene level, you need to aggregate the TPM values from Kallisto at the gene level instead of the transcript level. This is because Kallisto quantifies expression at the transcript level by default, meaning it gives TPM values for each transcript (isoform) of a gene.

To compare TPMs between Kallisto and Bowtie2, and compute log fold changes and other statistics for each gene, you will need to sum the TPM values for all transcripts of a gene (in case of multi-transcript genes) to get the overall expression of that gene.

- For Bowtie2-based output, calculate TPM values from the `counts.txt` file obtained in the previous step. You can use this function or construct your own.

```
# Function to calculate TPM from featureCounts output
library(dplyr)
```

```
calculate_TPM <- function(counts_file, output_file = "tpm_values.txt") {
  # Step 1: Load the raw counts file generated by featureCounts
  counts <- read.table(counts_file, header = TRUE, sep = "\t")
  colnames(counts)[colnames(counts) == "results.alignment_sorted.bam"] <- "counts"

  # Step 2: Calculate RPK (Reads Per Kilobase)
  # Convert gene length to kilobases (kb)
  counts$Length_kb <- counts$Length / 1000 # Convert length to kb
  counts$RPK <- counts$counts / counts$Length_kb

  # Step 3: Calculate CPM (Counts Per Million)
  # First, calculate the total reads in the sample
  total_reads <- sum(counts$counts)

  # Now calculate CPM for each gene
  counts$CPM <- (counts$counts / total_reads) * 1e6

  # Step 4: Calculate TPM (Transcripts Per Million)
  # Calculate TPM by normalizing CPM by gene length
  counts$TPM <- (counts$CPM / counts$Length_kb)
```

```
# Print the first few rows with TPM values
head(counts)

# Step 5: Save the result in a new file (optional)
write.table(counts, file = output_file, sep = "\t", row.names = FALSE,
            quote = FALSE)

return(counts) # Return the dataframe with TPM values
}
calculate_TPM("results/counts.txt", "results/tpm_values.txt")
```

Calculate TPM values for each condition and replicate.

Post quantification exercises

For the results of bowtie2 and Kallisto, perform the following steps for each:

- Grouped Boxplots of the three biological replicates per group: replicates all in the same color and side by side, in total six boxplots of each three in two colors
 - Do we need normalization? Are the general median, means, and standard deviations comparable?
 - * If you find you need normalization, apply it on the data, re-plot it and continue with the normalized data in the subsequent analysis
- Compute the log fold changes for all genes: Use the base 2 and compute how many times for each gene the expression is greater in one condition than the other. What formula do you need to use here?
 - Plot the distribution of all genes log fold changes
 - Find those genes that have a log fold change either ≥ 2 or ≤ -2
 - Look at those genes and report a few that you find interesting

Compare the results of Kallisto and Bowtie2:

- compute the differences of TPM values for each gene and plot them distribution of those diffs
- compute the fraction for each gene $\text{TPM(Kallisto)} / \text{TPM(Bowtie2)}$ take the log and plot the distribution of those logs
- compare the genes whose expression changed starkly ($\log \text{ fold change} \geq 2$ or ≤ -2), are these the same genes? Compute the Jaccard Similarity Coefficient of those two sets: `Stark_Changed_Genes(Bowtie)` vs `Stark_Changed_Genes(Kallisto)`

Enjoy!